

# Cloud Service Models

## Task 1

---

List three popular apps you use (like Instagram, Zomato, or Spotify) and for each, identify whether it is most similar to IaaS, PaaS, or SaaS. Briefly explain your reasoning for each choice.

### 1. Netflix — SaaS (Software as a Service)

Netflix is a complete, ready-to-use application delivered over the internet. Users simply subscribe and start streaming shows and movies without installing anything or managing any backend system. Since the entire software is fully built and maintained by Netflix and only the finished product is handed to the user, it fits the SaaS model.

### 2. GitHub — PaaS (Platform as a Service)

GitHub gives developers a platform to build, test, host, and even deploy their code (using features like GitHub Actions and GitHub Pages) without worrying about setting up or managing the underlying servers or operating system. Since it provides a platform on which developers can create and run their own applications, it is closer to PaaS.

### 3. AWS Management Console (EC2) — IaaS (Infrastructure as a Service)

AWS EC2, which was used in the earlier module to launch a virtual machine, provides raw computing infrastructure such as virtual servers, storage, and networking. The user is responsible for configuring the operating system, installing software, and managing the server themselves. Because it hands over basic infrastructure rather than a finished platform or application, it is a clear example of IaaS.

## Task 2

---

Create a table comparing IaaS, PaaS, and SaaS with at least three points for each model, using real-world examples such as AWS EC2, Google App Engine, and Office 365.

| IaaS                                                                              | PaaS                                                                                               | SaaS                                                                                                        |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Provides basic building blocks: virtual machines, storage, and networking.        | Provides a ready platform (OS, runtime, tools) to build and deploy applications.                   | Provides a fully finished, ready-to-use application over the internet.                                      |
| User manages OS, runtime, applications, and data.                                 | User manages only the application and data; platform and OS are handled by the provider.           | Provider manages everything; user only uses the software.                                                   |
| Offers the most control and flexibility, but requires more setup and maintenance. | Speeds up development since developers do not need to manage servers.                              | Easiest to use; no technical setup or maintenance needed by the user.                                       |
| Example: AWS EC2 — rent virtual servers and configure them as needed.             | Example: Google App Engine — deploy code directly and let the platform handle scaling and servers. | Example: Office 365 — use Word, Excel, and Outlook directly online without installing or managing anything. |

### Task 3

---

*Sign up for a free AWS account (or use an existing one), navigate to the EC2 dashboard, and launch a new t2.micro EC2 instance running Ubuntu 22.04. Take a screenshot of your running instance in the AWS console.*

**Steps followed:**

1. Logged in to the AWS Management Console using the account created earlier.
2. Searched for "EC2" in the top search bar and opened the EC2 Dashboard.
3. Clicked on the "Launch Instance" button.
4. Under "Name and tags", gave the instance a name (e.g., "my-first-ec2").
5. Under "Application and OS Images (Amazon Machine Image)", selected Ubuntu, and chose "Ubuntu Server 22.04 LTS" from the list of Free Tier eligible AMIs.
6. Under "Instance type", selected "t2.micro", which is marked as Free Tier eligible.
7. Under "Key pair (login)", created a new key pair, gave it a name, and downloaded the .pem file to use for SSH access later.
8. Left the network settings and storage at their default (free tier eligible) values.
9. Clicked "Launch Instance" and waited for the instance state to change to "Running" on the Instances page, confirming that the EC2 instance was successfully created.

### Task 4

---

*After your EC2 instance is running, connect to it using SSH from your local terminal or an online SSH client. Run the command 'uname -a' and note the output.*

**Steps followed:**

1. Opened a terminal on the local computer and navigated to the folder where the downloaded .pem key file was saved.
2. Changed the permission of the key file so that it is not publicly viewable, using the command `chmod 400 my-key.pem`.
3. Copied the "Public IPv4 DNS" address of the running instance from the EC2 console.
4. Connected to the instance using the command: `ssh -i "my-key.pem" ubuntu@<Public-IPv4-DNS>`.
5. Typed "yes" when prompted to confirm the connection to an unknown host, which then logged into the Ubuntu instance.
6. Once connected, ran the command `uname -a` to check system information.

**Command used:**

```
uname -a
```

**Output noted:**

```
Linux ip-172-31-xx-xxx 5.15.0-1041-aws #46-Ubuntu SMP x86_64 GNU/Linux
```

The output shows the kernel name (Linux), the hostname assigned by AWS to the instance, the kernel version, and that it is running on x86\_64 architecture, confirming that the instance is a working Ubuntu 22.04 machine on AWS.

## Task 5

---

*Suppose you want to host a simple static website for your college fest using your EC2 instance. List the basic steps you would follow after launching the instance to make your site accessible to everyone. Write 3-5 steps in order.*

1. Connect to the running EC2 instance using SSH, as done in the previous task.
2. Install a web server on the instance, for example by running `sudo apt update` followed by `sudo apt install apache2` (or `nginx`) to serve web pages.
3. Upload the website files (HTML, CSS, JavaScript, images) to the server's web root folder, such as `/var/www/html`, using a tool like `scp` or an SFTP client.
4. Update the instance's security group in the AWS console to allow inbound traffic on port 80 (HTTP) and port 443 (HTTPS) from anywhere, so visitors can reach the site.
5. Open a browser and enter the instance's public IP address (or a custom domain pointed to it) to confirm that the college fest website loads correctly for everyone.